

Technical Specification and Dataset Architecture for Training AI Code Reviewers: OWASP Top 10 Vulnerabilities (SQLi, XSS, and Broken Authentication)

Neural Detection Mechanics and Security Mapping

To train a state-of-the-art AI code reviewer for enterprise security, the underlying dataset must map syntactic and semantic structures directly to vulnerability classes.¹ Traditional regex engines miss the contextual data flow (taint analysis) that distinguishes safe execution from exploitable vulnerabilities.³ For instance, a model must understand whether user-controlled input (the source) flows into an execution sink (e.g., a raw database engine or a browser document object) without passing through a neutralizing sanitization function.⁴ During fine-tuning, the model can be evaluated as a token-level or block-level binary classifier. The loss function used to optimize the model on code security classification tasks is formulated using binary cross-entropy loss:

$$L = -\frac{1}{N} \sum_{i=1}^N [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)]$$

where y_i represents the ground-truth binary security label (0 for compliant, 1 for vulnerable) and p_i is the predicted probability of the code segment being vulnerable.

Target Vulnerability Coverage Matrix

OWASP Category	CWE Mapping	Primary Source / Input	Target Execution Sink	Optimal Remediation Strategy
A03:2021 Injection ⁶	CWE-89 (SQL Injection) ⁸	HttpServletRequest.getParameter() ³	Statement.executeUpdate() / MyBatis \${} ¹⁰	Parameterized PreparedStatements / MyBatis #{} ¹³

A03:2021 Injection ⁶	CWE-79 (Cross-Site Scripting) ⁸	HTTP Request payload / Persistent Database ¹⁴	Raw JSP scriptlet / HTML DOM innerHTML ¹⁵	Contextual output encoding / HTML Sanitization ³
A07:2021 Identification ¹⁸	CWE-384 (Session Fixation) ⁸	Pre-authenticated Session Cookie ¹⁹	Authenticated Servlet Context ¹⁹	Session ID regeneration on login ¹⁹
A07:2021 Identification ¹⁸	CWE-287 (Improper Authentication) ¹⁸	Plaintext password parameters ¹²	Domain persistence layer ¹²	Secure salted hashing (BCrypt) / Rate limiting ¹²

Vulnerability 1: SQL Injection (SQLi)

SQL Injection occurs when untrusted user input is directly concatenated into a dynamic SQL string and parsed as executable commands by the database engine.³

1. Plain JDBC Statement Concatenation

The standard Statement interface does not separate query logic from variable data.¹⁰

```

Java
// Non-Compliant: User input concatenated directly into a JDBC Statement
public List<User> getUserById(String userId) throws SQLException {
    Connection conn = dataSource.getConnection();
    Statement stmt = conn.createStatement();
    // Vulnerable: Injecting "1' OR '1'='1" bypasses execution logic
    String sql = "SELECT * FROM users WHERE user_id = '" + userId + "'"
    ResultSet rs = stmt.executeQuery(sql);
    return mapUsers(rs);
}

```

Java

// Compliant: Parameter binding using PreparedStatement separates logic from data

```
public List<User> getUserById(String userId) throws SQLException
```

```
    String sql = "SELECT * FROM users WHERE user_id =?"
```

```
    try (Connection conn = dataSource.getConnection())
```

```
        PreparedStatement pstmt = conn.prepareStatement(sql)
```

```
        pstmt.setString(1, userId) // Neutralizes active SQL characters
```

```
        try (ResultSet rs = pstmt.executeQuery())
```

```
            return mapUsers(rs)
```

2. Object-Relational Mapping (JPA / Hibernate)

Using an ORM like Hibernate or JPA does not guarantee security if developers concatenate raw values within the Java Persistence Query Language (JPQL) or Java API.¹³

Java

// Non-Compliant: Dynamic JPQL query constructed via dynamic string formatting

```
public List<Account> findAccounts(String customerId)
```

```
    String jql = "FROM Account WHERE customerId = '" + customerId + "'"
```

```
    TypedQuery<Account> query = entityManager.createQuery(jql, Account.class) //
```

Vulnerable to JPQL Injection

```
    return query.getResultList()
```

Java

// Compliant: JPQL query using structured setParameter binding

```
public List<Account> findAccounts(String customerId)
```

```
    String jql = "FROM Account WHERE customerId = :customerId"
```

```
    TypedQuery<Account> query = entityManager.createQuery(jql, Account.class)
```

```
        .setParameter("customerId", customerId); // Enforces parameterization
```

```
    return query.getResultList();
```

|

Java

// Compliant: Dynamic criteria queries via CriteriaBuilder API

```
public List<Account> findAccountsSecurely(String customerId)
```

```
    CriteriaBuilder cb = entityManager.getCriteriaBuilder();
```

```
    CriteriaQuery<Account> cq = cb.createQuery(Account.class);
```

```
    Root<Account> root = cq.from(Account.class);
```

```
    cq.select(root).where(cb.equal(root.get("customerId"), customerId)); // Safe programmatic  
building
```

```
    return entityManager.createQuery(cq).getResultList();
```

|

3. MyBatis Mapping Engine

In MyBatis, the string substitution syntax `${}` interpolates unmodified strings directly into queries, while the `#{}` syntax generates safe prepared statements.¹¹

XML

```
<select id="findByCode" resultType="com.enterprise.Product">
```

```
    SELECT * FROM products WHERE sku_code = ${code}
```

```
</select>
```

XML

```
<select id="findByCode" resultType="com.enterprise.Product">  
  SELECT * FROM products WHERE sku_code = #{code}  
</select>
```

Vulnerability 2: Cross-Site Scripting (XSS)

Cross-Site Scripting occurs when an application receives untrusted data and displays it back to the client browser without proper validation or contextual output encoding.³

1. Reflected XSS in JSP Output

Rendering request parameters directly via raw Expressions (<%= %> or \${}) in JavaServer Pages permits attackers to inject malicious script tags.¹⁵

Java

```
<div>  
  Welcome, <%= request.getParameter("username") %>  
</div>
```

Java

```
<%@ taglib prefix="c" uri="jakarta.tags.core" %>  
<div>  
  Welcome, <c:out value="${param.username}" />  
</div>
```

2. Context-Aware Output Encoding (OWASP Java Encoder)

Standard HTML escaping is insufficient when data is rendered inside specific contexts like

JavaScript blocks, HTML attributes, or CSS style blocks.¹⁶

```
Java
// Non-Compliant: Raw parameter rendering inside a JavaScript variable block
public void renderScriptBlock(HttpServletRequest request, HttpServletResponse response) throws
IOException
    String themeColor = request.getParameter("theme")
    response.getWriter().println "<script>"
    response.getWriter().println "var currentTheme = '" + themeColor + "';" // Vulnerable to
Javascript breakout
    response.getWriter().println "</script>"

```

```
Java
// Compliant: Enforcing contextual output encoding with the OWASP Java Encoder
import org.owasp.encoder.Encode

public void renderScriptBlock(HttpServletRequest request, HttpServletResponse response) throws
IOException
    String themeColor = request.getParameter("theme")
    response.getWriter().println "<script>"
    // Safe: Uses specific javascript block context encoder
    response.getWriter().println "var currentTheme = '"
    Encode.forJavaScriptBlock(themeColor) + "';"
    response.getWriter().println "</script>"

```

3. Rich Text HTML Sanitization (OWASP HTML Sanitizer)

If an application must permit users to submit formatted HTML markup, it must strip out active executable scripts using a strict, positive-whitelist policy sanitizer.¹⁷

Java

```
// Non-Compliant: Accepting and returning rich text directly opens stored XSS vectors
public String renderRichComment(String rawComment)
    return "<div>" + rawComment + "</div>" // Malicious events like onerror() will execute
```

Java

```
// Compliant: Strict element whitelisting via OWASP Java HTML Sanitizer
import org.owasp.html.HtmlPolicyBuilder
import org.owasp.html.PolicyFactory

public class ForumService
    private static final PolicyFactory POLICY = new HtmlPolicyBuilder()
        .allowCommonBlockElements()
        .allowCommonInlineFormattingElements()
        .allowElements("a")
        .allowUriProtocols("https")
        .allowAttributes("href", onElements("a"))
        .requireRelNofollowOnLinks()
        .toFactory()

    public String renderRichComment(String rawComment)
        String safeHTML = POLICY.sanitize(rawComment) // Strips <script>, events, and unlisted
        attributes
        return "<div>" + safeHTML + "</div>"
```

Vulnerability 3: Broken Authentication & Session Fixation

Broken Authentication encompasses failures in managing password complexity, rate limits,

session tokens, or session lifecycles.¹⁸ Session Fixation allows an attacker to hijack a session by pre-setting a valid session identifier on the victim's browser.¹⁹

1. Insecure Credential Storage

Storing user credentials in plain text or using weak, legacy algorithms (such as MD5 or SHA-1) allows easy extraction during a data leak.⁶

```
Java
// Non-Compliant: Storing password as plain text inside persistence layer
public void registerUser(String email, String plaintextPassword) throws SQLException
{
    try {
        Connection conn = dataSource.getConnection();
        PreparedStatement pstmt = conn.prepareStatement("INSERT INTO users(email,
password) VALUES(?,?)");
        pstmt.setString(1, email);
        pstmt.setString(2, plaintextPassword); // Highly vulnerable
        pstmt.executeUpdate();
    }
}
```

```
Java
// Compliant: Hash and salt passwords using BCrypt before persistence
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;

public class UserService
{
    private final BCryptPasswordEncoder encoder = new BCryptPasswordEncoder();

    public void registerUser(String email, String plaintextPassword) throws SQLException
    {
        String secureHash = encoder.encode(plaintextPassword); // Multi-round salted hashing
        try {
            Connection conn = dataSource.getConnection();
            PreparedStatement pstmt = conn.prepareStatement("INSERT INTO users(email,
password) VALUES(?,?)");
        }
    }
}
```

```

        pstmt.setString(1, email);
        pstmt.setString(2, secureHash);
        pstmt.executeUpdate();
    }
}

```

2. Manual Session ID Regeneration (Servlet API)

When a user logs in, the application must invalidate their existing, unauthenticated session and provision a new session ID to neutralize session fixation attacks.¹⁹

```

Java
// Non-Compliant: Performing user login without renewing the session ID
public void loginUser(HttpServletRequest request, String username)
{
    HttpSession session = request.getSession();
    session.setAttribute("user", username); // Attacker's pre-set JSESSIONID remains valid
}

```

```

Java
// Compliant: Invalidate session on authentication boundary, then provision a fresh session
public void loginUser(HttpServletRequest request, String username)
{
    HttpSession oldSession = request.getSession(false);
    java.util.Map<String, Object> attributes = new java.util.HashMap<>();

    if (oldSession != null)
    {
        // Extract temporary attributes before destruction
        java.util.Enumeration<String> keys = oldSession.getAttributeNames();
        while (keys.hasMoreElements())
        {
            String key = keys.nextElement();
            attributes.put(key, oldSession.getAttribute(key));
        }
    }
}

```

```

oldSession.invalidate(); // Destroys the pre-authentication session ID
}

HttpSession newSession = request.getSession(true) // Generates a fresh, randomized
session ID
attributes.forEach(newSession::setAttribute)
newSession.setAttribute("user", username);
}

```

3. Spring Security Session Policy Management

Spring Security provides declarative configuration chains to secure request paths and enforce automatic session ID regeneration on login.²²

```

Java
// Non-Compliant: Insecure endpoints and disabled session protection
@Configuration
@EnableWebSecurity
public class SecurityConfig
    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception
    {
        http
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/admin/**").permitAll() // Violates access control boundaries
                .anyRequest().permitAll()
            )
            .sessionManagement(session -> session
                .sessionFixation(sessionFixation -> sessionFixation.none()) // Vulnerable to fixation
            )
        return http.build();
    }
}

```

Java

```
// Compliant: Enforce path auth and declare session protection strategy
@Configuration
@EnableWebSecurity
public class SecurityConfig
{
    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception
    {
        http
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/admin/**").hasRole("ADMIN") // Restricts sensitive paths
                .anyRequest().authenticated() // Strict fail-closed policy
            )
            .sessionManagement(session -> session
                // Safe: Generates a new session and migrates temporary attributes
                .sessionFixation(sessionFixation -> sessionFixation.migrateSession())
            )
        ;
        return http.build();
    }
}
```

Static Code Analysis Rules and AST Specifications

The following PMD rules can be used to scan training repositories and validate code before ingestion into your dataset.¹¹

1. PMD Rule: Detect Dynamic SQL in JDBC Statements

This rule flags calls to `Statement.executeQuery()` or `Statement.executeUpdate()` where the query string is constructed dynamically.¹¹

XML

```
<rule name="AvoidDynamicSqlStatements" language="java" message="Use PreparedStatement
parameters instead of concatenating inputs into dynamic SQL query strings."
class="net.sourceforge.pmd.lang.rule.XPathRule">
    <description>atches JDBC Statements that construct SQL queries dynamically
exposing the application to SQL Injection </description>
    <priority> </priority>
    <properties>
```

```

    <property name="xpath">
      <value>
        [
          and
            PrimarySuffix/Arguments/ArgumentList/Expression//AdditiveExpression
          ]
      </value>
    </property>
  </properties>
</rule>

```

2. PMD Rule: Detect Loose Request Matcher Wildcards

This rule identifies Spring Security configurations that use global wildcards (/) within a permitAll() block, which can bypass access controls on nested administrative endpoints.¹¹

```

XML
<rule name="AvoidOverlyPermissiveRequestMatchers" language="java" message="Avoid mapping
global path wildcards to permitAll. Restrict access paths."
class="net.sourceforge.pmd.lang.rule.XPathRule">
  <description> Tags potentially insecure request matcher configuration chains where
sensitive sub-folders may be exposed </description>
  <priority> </priority>
  <properties>
    <property name="xpath">
      <value>
        [
          and
            PrimarySuffix/Arguments/ArgumentList/Expression/PrimaryExpression/PrimaryPrefix/Literal
            /contains(@Image, '/')//ancestor::PrimaryExpression
          ]
      </value>
    </property>
  </properties>
</rule>

```

Master Dataset Schema Specification (JSONL Format)

Each entry in your training dataset should be formatted as a single-line JSON Lines (JSONL) object to support efficient parsing during SFT (Supervised Fine-Tuning) epochs.¹⁸

JSON

```
{
  "instance_id" "MSR_OWASP_2026_SQLI_004"
  "metadata"
    "problem_domain" "Application Security"
    "difficulty" "Medium"
    "targeted_vulnerability" "CWE-89 SQL Injection"
    "pmd_rule_reference" "category/java/security.xml/AvoidDynamicSqlStatements"
  "context"
    "repository" "spring-projects/spring-data-jpa"
    "jdk_version" "Java 21"
    "problem_statement" "The repository search method constructs dynamic native query strings
using unvalidated parameters, permitting unauthorized database read/write actions."
    "original_code" "package com.enterprise.dao;\n\nimport com.enterprise.model.User;\nimport
jakarta.persistence.EntityManager;\nimport jakarta.persistence.Query;\nimport
java.util.List;\n\npublic class UserSearchDAO {\n  private final EntityManager em;\n\n  public
UserSearchDAO(EntityManager em) {\n    this.em = em;\n  }\n\n  @SuppressWarnings(\"unchecked\")\n  public List<User> searchByRole(String roleName) {\n
String rawQuery = \"SELECT * FROM users WHERE active = true AND role = '\" + roleName + '\"";\n
Query query = em.createNativeQuery(rawQuery, User.class);\n    return query.getResultList();\n
}\n}"
    "refactored_code" "package com.enterprise.dao;\n\nimport com.enterprise.model.User;\nimport
jakarta.persistence.EntityManager;\nimport jakarta.persistence.Query;\nimport
java.util.List;\n\npublic class UserSearchDAO {\n  private final EntityManager em;\n\n  public
UserSearchDAO(EntityManager em) {\n    this.em = em;\n  }\n\n  @SuppressWarnings(\"unchecked\")\n  public List<User> searchByRole(String roleName) {\n
String parameterizedQuery = \"SELECT * FROM users WHERE active = true AND role = :role\";\n
Query query = em.createNativeQuery(parameterizedQuery, User.class);\n
.setParameter(\"role\", roleName);\n    return query.getResultList();\n  }\n}"
    "structured_review"
    "severity" "Critical"
}
```

```
"line_range" 24 25
"diagnostic" "Unvalidated dynamic string concatenation inside createNativeQuery."
"remediation_justification" "Direct string interpolation permits attackers to alter the query logic
(e.g. escaping the query to dump records). Using named parameter binding separates the SQL
query structure from variables."

```

JSON

```
"instance_id" "MSR_OWASP_2026_XSS_087"
"metadata"
"problem_domain" "Application Security"
"difficulty" "Hard"
"targeted_vulnerability" "CWE-79 Cross-Site Scripting"
"pmd_rule_reference" "category/java/security.xml/XssUnencodedOutput"
"context"
"repository" "apache/struts-examples"
"jdk_version" "Java 17"
"problem_statement" "The custom servlet outputs user input directly inside an HTML body
without validation, leading to Reflected Cross-Site Scripting."
"original_code" "package com.enterprise.servlet;\n\nimport java.io.IOException;\nimport
jakarta.servlet.http.*;\n\npublic class FeedbackServlet extends HttpServlet {\n    protected void
doPost(HttpServletRequest req, HttpServletResponse resp) throws IOException {\n        String
feedback = req.getParameter(\"feedbackText\");\n        resp.setContentType(\"text/html\");\n
resp.getWriter().println(\"<html><body>\");\n        resp.getWriter().println(\"<h1>Your feedback has
been logged:</h1>\");\n        resp.getWriter().println(\"<div class='feedback-block'>\") + feedback +
\"</div>\");\n        resp.getWriter().println(\"</body></html>\");\n    }\n}"
"refactored_code" "package com.enterprise.servlet;\n\nimport
org.owasp.encoder.Encode;\nimport java.io.IOException;\nimport jakarta.servlet.http.*;\n\npublic
class FeedbackServlet extends HttpServlet {\n    protected void doPost(HttpServletRequest req,
HttpServletResponse resp) throws IOException {\n        String feedback =
req.getParameter(\"feedbackText\");\n        resp.setContentType(\"text/html\");\n
resp.getWriter().println(\"<html><body>\");\n        resp.getWriter().println(\"<h1>Your feedback has
been logged:</h1>\");\n        resp.getWriter().println(\"<div class='feedback-block'>\") +
```

```

Encode.forHtml(feedback) + "</div>");\n    resp.getWriter().println("</body></html>");\n    }\n}"
"structured_review"
{
  "severity" "High"
  "line_range" 12 12
  "diagnostic" "Unescaped user input appended directly to output stream."
  "remediation_justification" "Rendering raw user parameters within HTML tags allows attackers
to inject executable elements. Using contextual encoding like Encode.forHtml converts special
characters into safe HTML entities."
}
}
}

```

JSON

```

{
  "instance_id" "MSR_OWASP_2026_FIX_012"
  "metadata"
  {
    "problem_domain" "Application Security"
    "difficulty" "Medium"
    "targeted_vulnerability" "CWE-384 Session Fixation"
    "pmd_rule_reference" "category/java/security.xml/AvoidSessionFixation"
  }
  "context"
  {
    "repository" "spring-projects/spring-security-samples"
    "jdk_version" "Java 21"
    "problem_statement" "Upon authentication validation, the system retains the existing user
session ID, allowing an attacker with access to the pre-auth session ID to hijack the authenticated
context."
  }
  "original_code" "package com.enterprise.security;\n\nimport jakarta.servlet.http.*;\n\npublic class
CustomSessionManager {\n    public void authorizeUser(HttpServletRequest request, String
verifiedUser) {\n        HttpSession session = request.getSession();\n
session.setAttribute(\"USER_PRINCIPAL\", verifiedUser);\n        session.setAttribute(\"AUTH_STATUS\",
\"CONFIRMED\");\n    }\n}"
  "refactored_code" "package com.enterprise.security;\n\nimport jakarta.servlet.http.*;\nimport
java.util.*;\n\npublic class CustomSessionManager {\n    public void
authorizeUser(HttpServletRequest request, String verifiedUser) {\n        HttpSession oldSession =
request.getSession(false);\n        Map<String, Object> sessionData = new HashMap<>();\n        if

```


- <https://sites.google.com/view/writing-custom-pmd-rules-52/>
11. Code Style | PMD Source Code Analyzer, accessed on June 12, 2026, https://pmd.github.io/pmd/pmd_rules_java_codestyle.html
 12. 15-121 Java Style Guide - Carnegie Mellon University in Qatar, accessed on June 12, 2026, <https://web2.qatar.cmu.edu/cs/15121/resources/styleguide/>
 13. SmellBench: Towards Fine-Grained Evaluation of Code Agents on Refactoring Tasks - arXiv, accessed on June 12, 2026, <https://arxiv.org/pdf/2606.05574>
 14. Home · microsoft/CodeXGLUE Wiki · GitHub, accessed on June 12, 2026, <https://github.com/microsoft/CodeXGLUE/wiki>
 15. Source code review: A comprehensive guide to secure development - Sonar, accessed on June 12, 2026, <https://www.sonarsource.com/resources/library/source-code-review/>
 16. CodeXGLUE: A Machine Learning Benchmark Dataset for Code Understanding and Generation - arXiv, accessed on June 12, 2026, <https://arxiv.org/pdf/2102.04664>
 17. microsoft/CodeXGLUE - GitHub, accessed on June 12, 2026, <https://github.com/microsoft/codexglue>
 18. github/CodeSearchNet: Datasets, tools, and benchmarks for representation learning of code., accessed on June 12, 2026, <https://github.com/github/CodeSearchNet>
 19. The 3 Rules of Clean Java Code (That Most Devs Break) - YouTube, accessed on June 12, 2026, <https://www.youtube.com/watch?v=mrE399ajzs>
 20. anti-pattern · GitHub Topics, accessed on June 12, 2026, <https://github.com/topics/anti-pattern?o=asc&s=forks>
 21. jolemon/defects4j-dataset - GitHub, accessed on June 12, 2026, <https://github.com/jolemon/defects4j-dataset>
 22. ajibawa-2023/Java-Code-Large · Datasets at Hugging Face, accessed on June 12, 2026, <https://huggingface.co/datasets/ajibawa-2023/Java-Code-Large>
 23. HuggingFaceH4/Code-Feedback · Datasets at Hugging Face, accessed on June 12, 2026, <https://huggingface.co/datasets/HuggingFaceH4/Code-Feedback>